

Greatest Common Divisor (GCD) of Two Numbers

Abdulla

23/07/2026

1 Underlying Theorem

Theorem 1. *For any integers a and b ,*

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Proof. By Euclid's Division Algorithm,

$$a = bq + r, \quad 0 \leq r < b, \tag{1}$$

where

$$r = a \bmod b.$$

Let d be a common divisor of a and b . Since $d \mid a$ and $d \mid b$, we have

$$d \mid (a - bq).$$

Since

$$a - bq = r,$$

it follows that $d \mid r$. Hence, d is also a common divisor of b and r .

Conversely, let d be a common divisor of b and r . Since

$$a = bq + r,$$

we have $d \mid a$. Thus, d is also a common divisor of a and b .

Therefore, the set of common divisors of (a, b) is identical to the set of common divisors of (b, r) . Hence,

$$\boxed{\gcd(a, b) = \gcd(b, r) = \gcd(b, a \bmod b).}$$

□

The above theorem can be applied repeatedly to compute the GCD of two integers efficiently.

2 Algorithm

The following C program implements Euclid's algorithm.

```
1 #include <stdio.h>
2
3 unsigned int gcd(unsigned int a, unsigned int b)
4 {
5     if (b == 0)
6         return a;
7
8     unsigned int rem = a % b;
9
10    while (rem != 0)
11    {
12        a = b;
13        b = rem;
14        rem = a % b;
15    }
16
17    return b;
18 }
19
20 int main()
21 {
22     printf("%u\n", gcd(18, 12));
23     return 0;
24 }
```

3 Running Time Analysis

Let the sequence of remainders generated by Euclid's algorithm be

$$r_0, r_1, r_2, \dots$$

where

$$r_0 = q_1 r_1 + r_2.$$

Substituting

$$r_1 = q_2 r_2 + r_3,$$

gives

$$r_0 = q_1(q_2 r_2 + r_3) + r_2 = (q_1 q_2 + 1)r_2 + q_1 r_3.$$

Since $q_1, q_2 \geq 1$ and $r_3 > 0$ (unless the algorithm has already terminated),

$$r_0 > 2r_2,$$

which implies

$$r_2 < \frac{r_0}{2}.$$

Thus, after two iterations, the remainder is reduced by at least a factor of two. Applying the same argument repeatedly to the pairs

$$(r_2, r_3), (r_4, r_5), \dots,$$

shows inductively that the remainder loses at least one bit every two iterations.

Therefore, for inputs of bit-length n , the algorithm performs at most $2n$ iterations, i.e., the number of iterations is $O(n)$.

Each iteration performs one modulo operation. Using the schoolbook division algorithm, computing the modulo of two n -bit integers requires $O(n^2)$ time. Since there are $O(n)$ iterations, the overall running time is

$$O(n) \times O(n^2) = O(n^3).$$